



Academic Year: 2026-27

Semester: VII

Class / Branch/ Div: BE- IT/B

Subject: SAD Lab

Name of Instructor: Prof. Ganesh Gourshete

Name of Student: Sonika Sawant

Student ID: 23104054

Roll No. 65

Date of Submission: 21-07-26

Experiment No:3

Aim: To study and demonstrate the use of SAST Tool.

Step 1: To get sample vulnerable code of python code use GitHub as shown below.

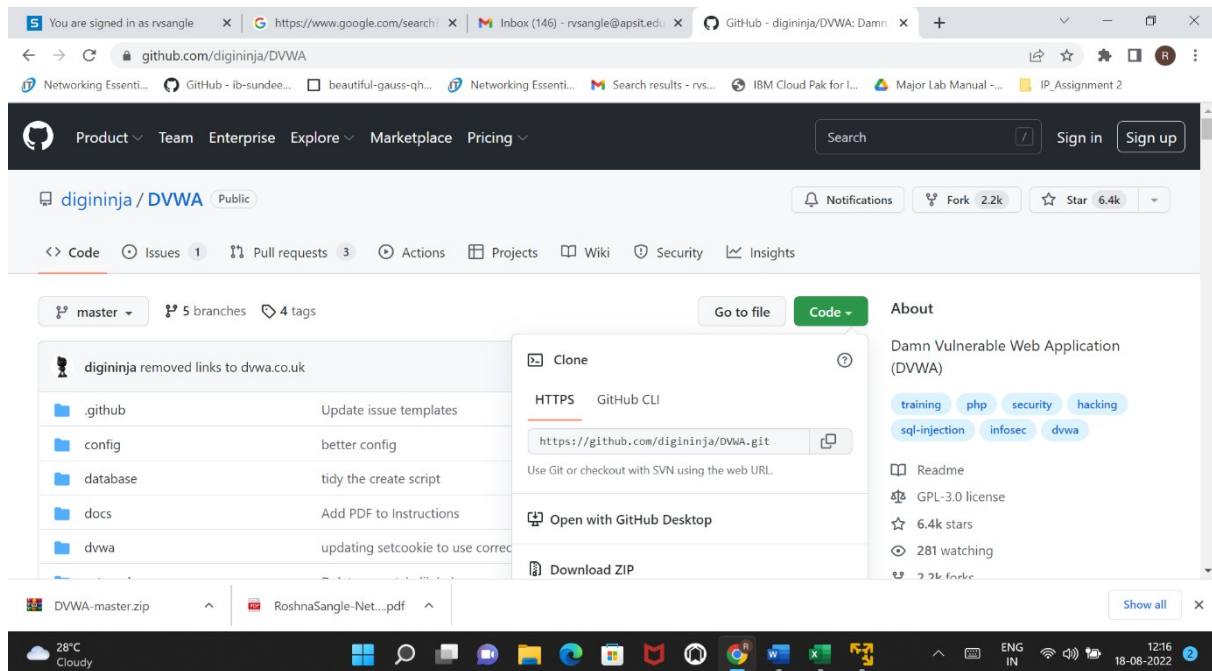
Example :- vulpy

The screenshot shows the GitHub repository page for 'fportantier / vulpy'. The repository is public and has 526 forks and 128 stars. The repository description is 'Vulnerable Python Application To Learn Secure Development'. The repository contains several files and folders, including 'bad', 'good', 'utils', '.gitignore', 'LICENSE', 'README.rst', 'install-on-kali.sh', and 'owasp-asvs-4.0.csv'. The repository was created by Fabian Martinez Portantier 6 years ago and has 58 commits.

File/Folder	Description	Created
bad	improvements	7 years ago
good	improvements	7 years ago
utils	improvements	7 years ago
.gitignore	background images. gitignore sqlite files	7 years ago
LICENSE	Create LICENSE	7 years ago
README.rst	new origin repo (portantier -> fportantier)	6 years ago
install-on-kali.sh	new script	7 years ago
owasp-asvs-4.0.csv	improvements	7 years ago



Step 2: Either download the sample code zip file from GitHub or copy folder link as shown below.



Step 3: Once sample python code is copied make its clone using following command in terminal.

```
apsit@apsit-HP-280-G3-MT:~$ git clone https://github.com/fportantier/vulpy.git
Cloning into 'vulpy'...
remote: Enumerating objects: 437, done.
remote: Total 437 (delta 0), reused 0 (delta 0), pack-reused 437 (from 1)
Receiving objects: 100% (437/437), 2.90 MiB | 4.46 MiB/s, done.
Resolving deltas: 100% (223/223), done.
apsit@apsit-HP-280-G3-MT:~$
```

Step 4: Now check the files present in path using ls.

```
roshna@ubuntu:~/vulpy$ ls
bad good install-on-kali.sh LICENSE owasp-asvs-4.0.csv README.rst requirements.txt utils Vulnerable-Code-Snippets
```

Step 5: Now to get path of vulpy use pwd command.



PARSHVANATH CHARITABLE TRUST'S

A. P. SHAH INSTITUTE OF TECHNOLOGY

Department of Information Technology

(NBA Accredited)



```
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$ pwd
/home/apsit/vulpy/bad
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$
```

Step 6: Now test the given python code with bandit as shown below.

```
33 app.run(debug=True, host= '127.0.1.1', port=5000, extra_files= csp.txt)
-----

Code scanned:
  Total lines of code: 495
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0.0
    Low: 3.0
    Medium: 0.0
    High: 2.0
  Total issues (by confidence):
    Undefined: 0.0
    Low: 0.0
    Medium: 2.0
    High: 3.0
Files skipped (0):
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$
```



PARSHVANATH CHARITABLE TRUST'S

A. P. SHAH INSTITUTE OF TECHNOLOGY

Department of Information Technology

(NBA Accredited)



Bandit Error	Issue ID	Severity	Location	Cause	Recommended Fix
Importing <code>subprocess</code>	B404	Low	<code>brute.py:3</code>	The <code>subprocess</code> module can execute external programs, which may introduce security risks if used improperly.	Avoid <code>subprocess</code> if a Python library can perform the task. If it is required, validate all inputs and restrict executable paths.
<code>subprocess.run()</code> with external input	B603	Low	<code>brute.py:21</code>	Executing a program using variables (<code>program</code> , <code>username</code> , <code>password</code>) may lead to execution of untrusted input.	Validate and sanitize all user-controlled inputs. Use absolute executable paths, avoid passing untrusted values, and consider allowing-listing permitted commands.
Empty <code>except</code> block (<code>except: pass</code>)	B110	Low	<code>libsession.py:21</code>	Exceptions are silently ignored, making debugging difficult and potentially hiding security issues.	Catch specific exceptions (e.g., <code>ValueError</code> , <code>json.JSONDecodeError</code>) and log or handle them appropriately instead of using <code>pass</code> .
Flask running with <code>debug=True</code>	B201	High	<code>vulpy-ssl.py:29</code>	Flask debug mode exposes the interactive debugger, which can allow arbitrary code execution if accessible.	Disable debug mode in production: <code>app.run(debug=False, host='127.0.0.1', ssl_context=...)</code> or configure debug via environment variables (<code>FLASK_DEBUG=0</code>).
Flask running with <code>debug=True</code>	B201	High	<code>vulpy.py:55</code>	Same issue as above. The Werkzeug debugger is enabled.	Change to <code>app.run(debug=False, host='127.0.0.1', port=5000)</code> or control debug mode through application configuration.

Conclusion:

Code should be clean and safe. Vulnerabilities and bugs in software under development constitute a major security problem. Thus, we have mitigated similar kind of security risk associated with Python code using bandit as a SAST tool efficiently.